CHAPTER 10- JavaScript

10.1 Course Objective

- This material will take you through concepts of JavaScript. After successful completion of this course the associate should be able to use basic JavaScript functionality.

10.2 Course Content

10.2.1 Introduction

JavaScript is a scripting language that is used in most of the modern HTML pages.
It gives a dynamic touch to them.

JavaScript is used to change the content of HTML elements dynamically. It can also be used to read the content of HTML elements and validate them and eventually show corresponding message to the user.

Just like we have tags for HTML elements, JavaScript is written inside <script></script> tags. By convention script tags are placed inside head tag, although it is not necessary to do that.

Below is a sample of JavaScript code, in which an alert method is used which is used to display messages as pop-up.

```
<html>
    <head>
        <script>
            alert('Hello');
        </script>
    </head>
    <body>
    </body>
</html>
```

<u>OUTPUT:</u>

When the browser loads the web page, the scripts are also loaded. Thus in the example above, as soon as the web page is loaded the browser ran the script and the alert method was executed.

The JavaScript can be included into the web page in two ways.
We can either write the code in script tag or write the code in another external file, save it with **.js** extension and include in the web page.

For example we can write “alert('hello');” in a text file and save it as myfile.js
Then we can include the js file in the web page.

<script src=”myfile.js”></script>

10.2.2 Variables

Variables are used to store data and manipulate it, just like we had in Java.
A variable in JavaScript is declared as:
var variableName=data;

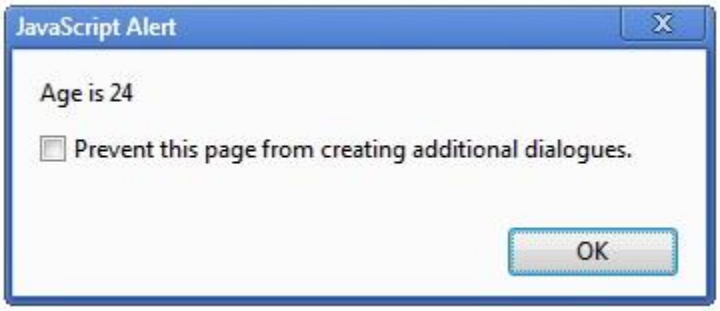
To declare a variable, “**var**” keyword is used. The data type of the variables is determined by the type of data they hold.

eg:

```
var x=”George”; //string
x='a';          //char
x=5;            //int
x=4.5           //float
x=true;         //boolean
x=new Array(2,4,6,3);   //array of integer
```

```
<html>
    <head>
        <script>
            var x="George";
            alert('Name is '+x);
            x=24;
            alert('Age is '+x);
        </script>
    </head>
    <body>
    </body>
</html>
```

OUTPUT:



Here we can see that same variable was able to store different values at different times. So java script is type independent.

10.2.3 Events

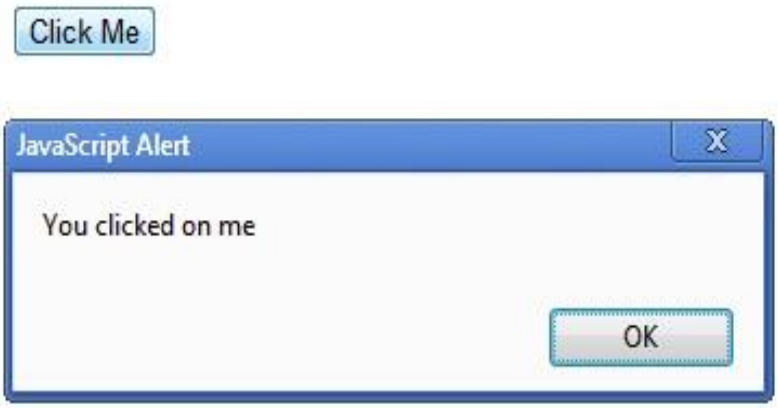
An event is defined as an action performed over an HTML element.

On various actions performed by mouse or keyboard JavaScript code can be called.
We will discuss about four basic events. **onclick, onsubmit, onchange, onselect**

onclick event occurs when we click on any HTML element. lets see an example of clicking a button.

```
<html>
    <head>
    </head>
    <body>
        <button onclick="alert('You clicked on me')">Click Me</button>
    </body>
</html>
```

OUTPUT:



When we click on a button, **onclick** event occurrs and the alert method is called as per the code.

onsubmit event occurs when the form is being submitted. Here is a sample code.

```
<html>
    <head>
    </head>
    <body>
        <form action="" method="post" onsubmit="alert('You are about to submit the form')">
        <input type="text">
        <input type="submit">
        </form>
    </body>
</html>
```

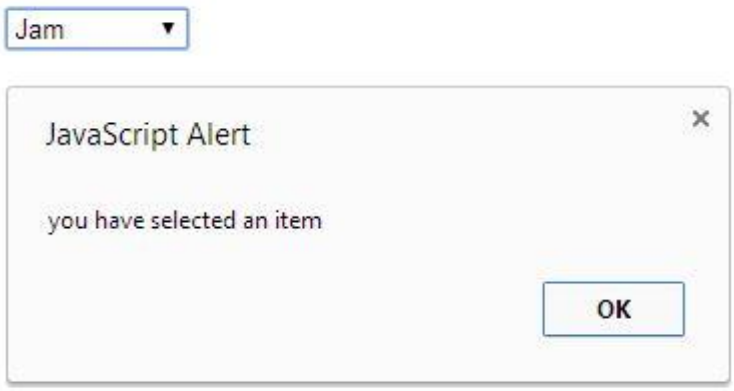
<u>OUTPUT:</u>

When submit button is pressed an **onsubmit** event occurs.

onchange event occurs whenever there is a change in the content of the particular element. For example:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<select  onchange="alert('you have selected an item')">
<option selected="selected">Select Item</option>
<option>Jam</option>
<option>Sauce</option>
<option>cream</option>
</select>
</body>
</html>
```

OUTPUT:

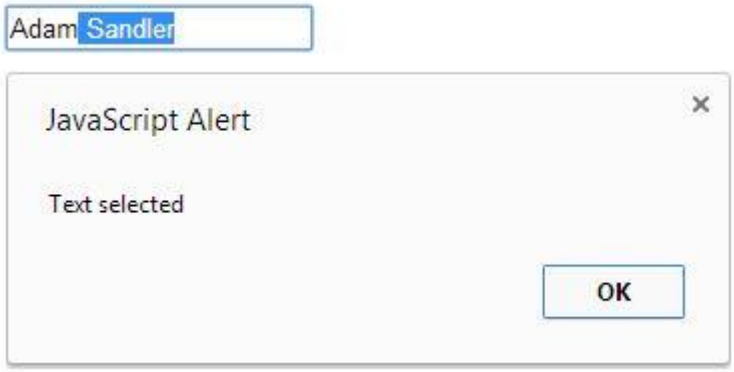


---

onselect event occurs when text of an input tag is selected. Sample code:

```
<html>
<head>
</head>
<body>
<input type="text" onselect="alert('Text selected')">
</body>
</html>
```

OUTPUT:



When the text in the text box is selected **onselect** event occurs and according to the code the alert method was called.

10.2.4 Functions

All java script codes written inside script tag gets executed after the page is loaded by the browser. If we want to execute specific functionality on various events we should put the JavaScript codes inside functions and call those function at the occurrence of events.

You must be familiar with method calling in Java. It is similar to that.
To write a function , the key word **"function"** is used, followed by the function name and the arguments.

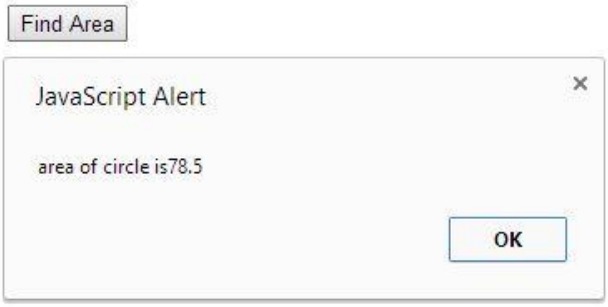
```
funciton myFunction(argument1, argument2....){

  var value=10;
  return value;
}
```

A variable declared inside a function becomes a local variable and thus can be used within the function only. A variable declared outside a function becomes a global variable and thus can be accessed by any function.

Sample Code:

```html
<html>
<head>
    <script>
        var pi=3.14;

        function calculateArea(radius){
            alert('area of circle is'+(pi*radius*radius));
        }
    </script>
</head>
<body>
    <button onclick="calculateArea(5)">Find Area</button>
</body>
</html>
```

OUTPUT:

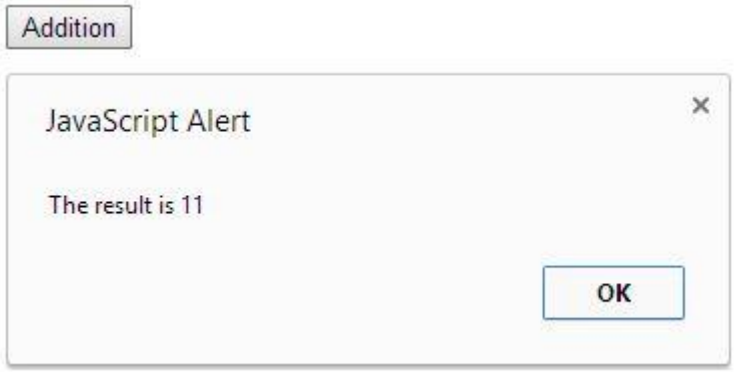


In the above example, a logic for calculating area of a circle is placed. Then on the event of clicking on the button, the function was called and a number 5 was passed as an argument. The area of the circle was shown through an alert box.

A function can call another function and return value to the calling function.
Here is an example.

```html
<html>
<head>
    <script>
        function calculate(x,y,z){
            var r=0;
            if(z==1)
            r=add(x,y);
            alert('The result is '+r);
        }

        function add(x,y){
            return x+y;
        }
    </script>
</head>
<body>
    <button onclick="calculate(5,6,1)">Addition</button>
</body>
</html>
```

OUTPUT:



In this example add function was called by the calculate function and it returned the sum the numbers.

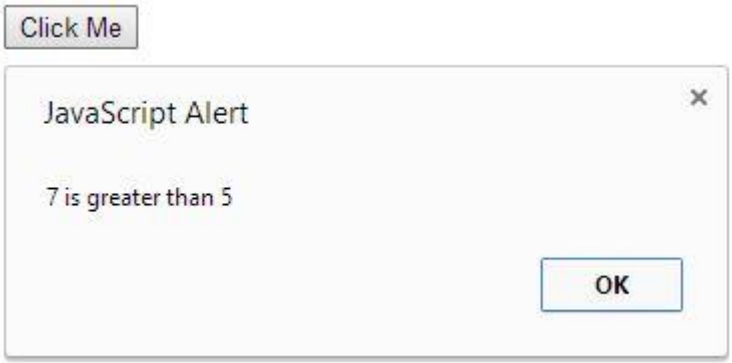
10.2.5 Conditional Operations

To compare between two expressions we need to do conditional operations.JavaScript support conditional operations like if-else, if-esle if and nested ifelse

Below is an example of if-else if :

```html
<html>
    <head>
        <script>
            function compare(x,y){
            if(x>y){
                alert(x+' is greater than '+y);
                }
            else if(x<y){
                alert(y+' is greater than '+x);
                    }
            else{
                alert('both the numbers are equal');
                }
                        }
        </script>
    </head>
    <body>
        <button type="button" onclick="compare(5,7)">Click Me</button>
    </body>
</html>
```

OUTPUT:



Here two numbers are compared and it is determined which number is greater.

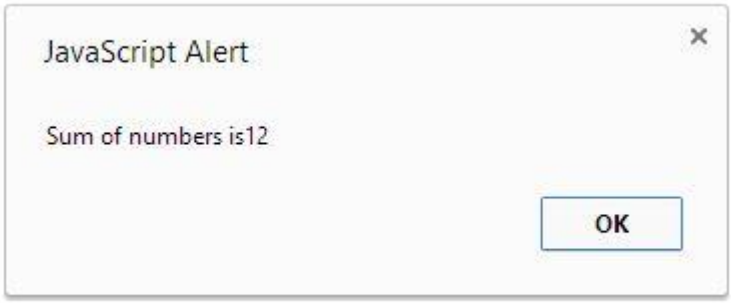
10.2.6 For Loop

In JavaScript there are many ways to iterate, one of them is to use a for loop.It is used in the same way as in Java.

Sample Code:

```
<html>
<head>
    <script>
    var x=new Array(2,4,6);
    var sum=0;
    for(var i=0;i<x.length;i++){
        sum=sum+x[i];
    }
    alert('Sum of numbers is'+sum);
    </script>
</head>
<body>
</body>
</html>
```

OUTPUT:



Here we iterated through the array of integers and calculated the sum of those numbers.The alert method showed the result on the screen.

This can also be achieved by placing the code inside a function and then calling the function through an event on HTML element.

10.2.7 PopUp Boxes

In JavaScript we can create small pop-up windows to either show message or get input from user. JavaScript provides three type of popup boxes:

Alert Box, **Confirm Box** and **Prompt Box**.

You have already seen alert box functionality, it is used to show messages as pop up.

Confirm box, as the name suggests, is used for confirmations. It returns either true or false when we click on Ok and Cancel respectively.

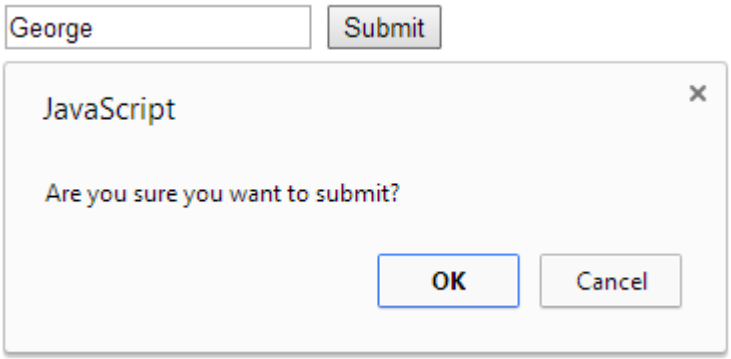
Syntax: Event=”return confirm(Message)”;

when we write **return** the boolean value returned will determine whether the form will be submitted or not.

Sample Code:

```
<html>
<head>
</head>
<body>
    <form action="/myservlet.do">
        <input type="text">
        <input type="submit" onclick="return confirm('Are you sure you want to submit?')">
    </form>
</body>
</html>
```

<u>OUTPUT:</u>



When you click on “Cancel”, false will be returned and the form will not be submitted.
When you click on “OK”, true will be returned and the form will proceed with the submission.

Prompt Box is used when we want an input from the user through the popup box.

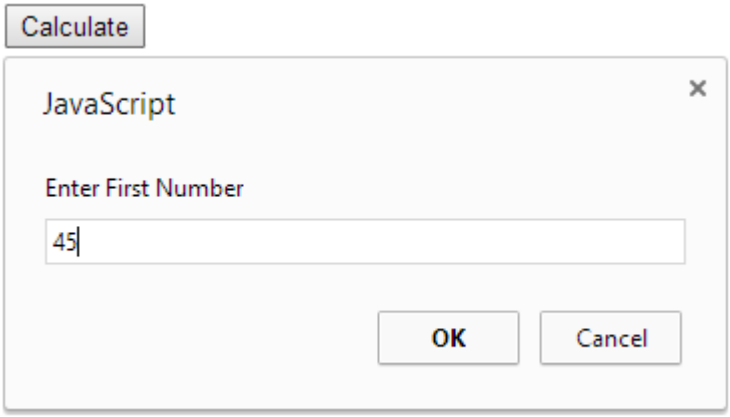
Syntax: prompt(Message,default value);

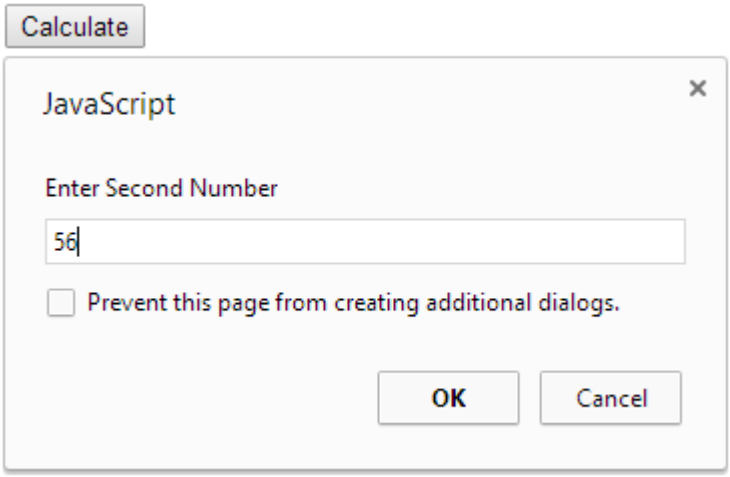
The default value is not necessary to give and can be skipped. The return type is string that we enter in the box.

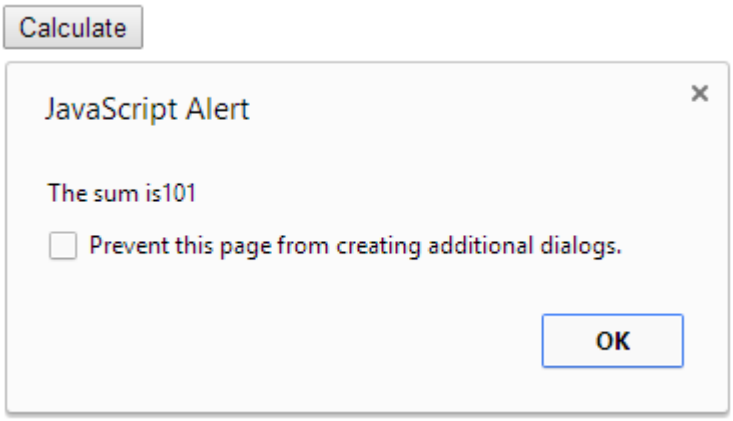
Sample code:

```
<html>
<head>
</head>
<script >
    function add(){
        var x=prompt("Enter First Number");
        var y=prompt("Enter Second Number");
        alert('The sum is'+(parseInt(x)+parseInt(y)));
    }

</script>
<body>
<button onclick="add()">Calculate</button>
</body>
</html>
```

OUTPUT:



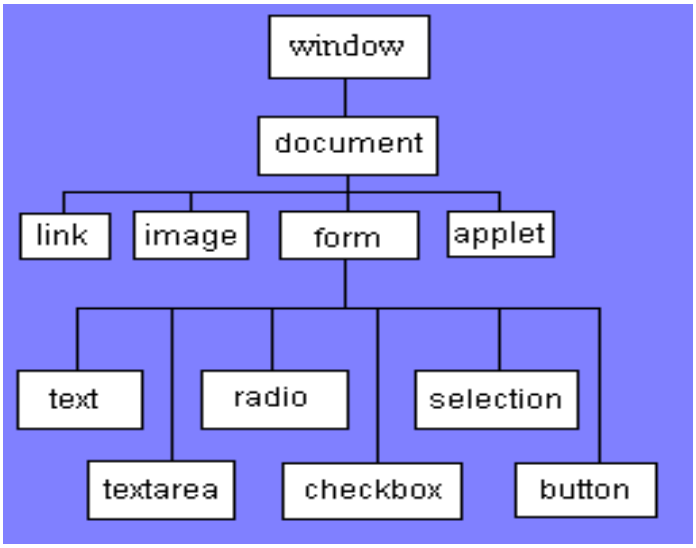




In the code we used inbuilt function parseInt() to convert string into int.
When Cancel is pressed on a prompt box, it returns null.

10.2.8 Document Object Model

When a browser loads a web page, it creates the Document Object Model of the page. Its a tree of HTML elements and we use it to traverse to various elements of the HTML page.



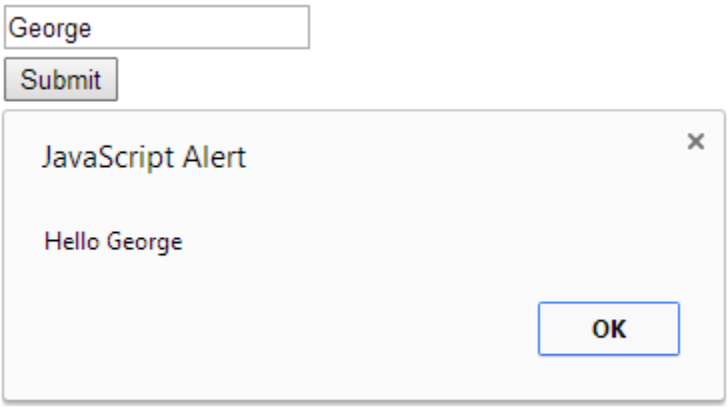
In the JavaScript function we can access the elements of the page either by traversing through the elements or by using some of the methods of these objects.

getElementById:

This method is used to access an element using the unique id we assign to it while creating the page.
Sample Code:

```
<html>
<head>
</head>
<script >
    function sayHello(){
        var x=document.getElementById("name");
        alert('Hello '+x.value);
    }

</script>
<body>
<form name="myForm" action="/register.do">
    <input name="firstName" id="name">
    <br>
    <input type="submit" onclick="sayHello()">
</form>
</body>
</html>
```

OUTPUT:



In the above example **var** x=document.getElementById("name") was used to access the text field having id=name.
Here the **document** object's method **getElementById** was used.
The same could be achieved by writing

var x=document.forms["myForm"]["name"].value where we use the 2D form of document model.

or

var x=document.myForm.firstName.value where we cascade through elements using their names.

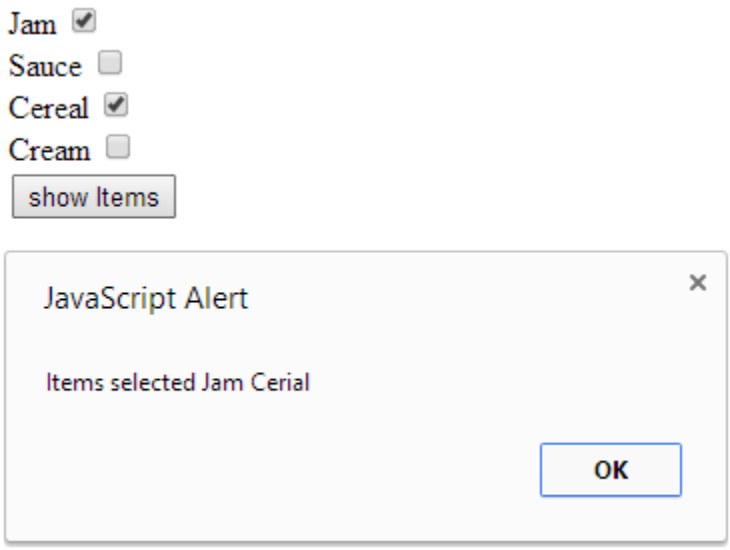
getElementsByName:

This method is used to get all the elements with the given name.

Sample code:

```
<html>
<head>
    <script>
    function printItems(){

        var x=document.getElementsByName("name");
        var items="";
        for(var i=0;i<x.length;i++){
            if(x[i].checked)
                items=items+x[i].value+" ";
        }
        alert('Items selected '+ items);
    }
    </script>
</head>
<body>
<form action="" name="myForm">
    Jam <input type="checkbox" name="name" value="Jam"><br>
    Sauce <input type="checkbox" name="name" value="Sauce"><br>
    Cereal <input type="checkbox" name="name" value="Cerial"><br>
    Cream <input type="checkbox" name="name" value="Cream"><br>
    <button onclick="printItems()">show Items</button>
</form>
</body>
</html>
```

OUTPUT:



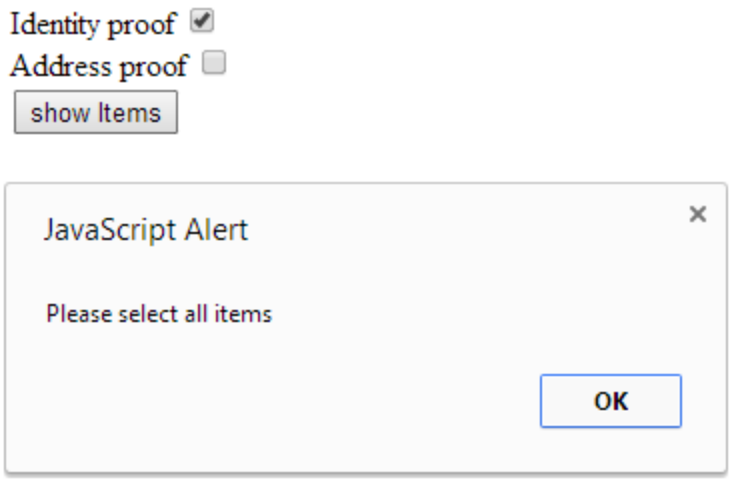
Here all the elements by that name are stored as array of elements in variable x. After that we can we can iterate through x and get all the elements and their value.

getElementsByTagName:

This method is used to get all the elements with the name of the tag.
Sample code:

```
<html>
<head>
    <script>
    function printItems(){

        var x=document.getElementsByTagName("input");
        var bool=true;
        for(var i=0;i<x.length;i++){
            if(!x[i].checked){
                bool=false;
                break;}
        }
            if(!bool)
        alert('Please select all items');
    }
    </script>
</head>
<body>
    Identity proof <input type="checkbox" name="name" value="Identity proof"><br>
    Address proof <input type="checkbox" name="name" value="Address proof"><br>
    <button onclick="printItems()">Proceed</button>
</body>
</html>
```

OUTPUT:



Here all the elements by the Tag name “input” are stored as array of elements in variable x. After that we can we can iterate through x and get all the elements and check whether they are checked or not.

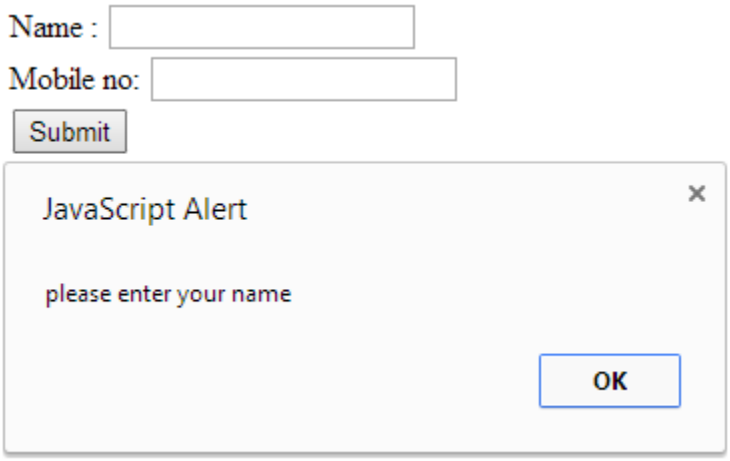
10.2.9 Validations

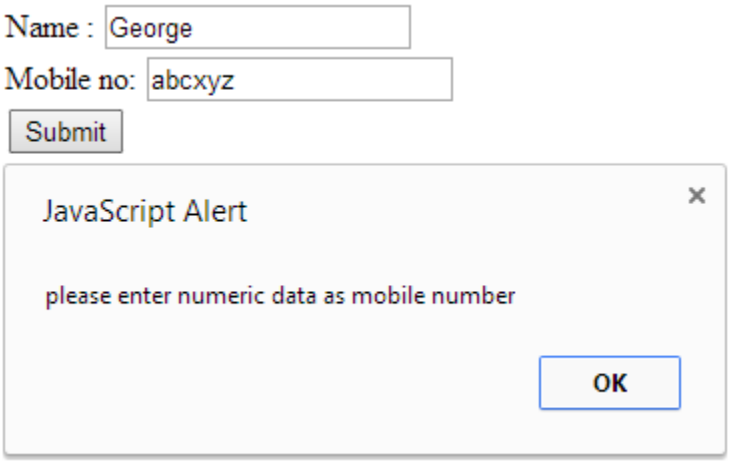
We can use JavaScript to validate the data entered by the users. For instance, we can check whether user has entered into the text field or not, whether user has enter a string in a number field, whether user has exceeded the world limit etc.

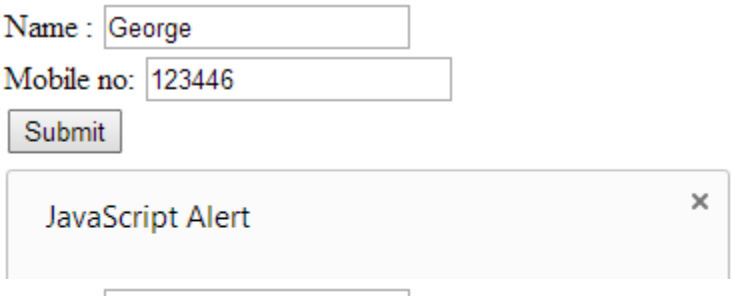
In the example below we will be validating a form based on the criteria mentioned above.

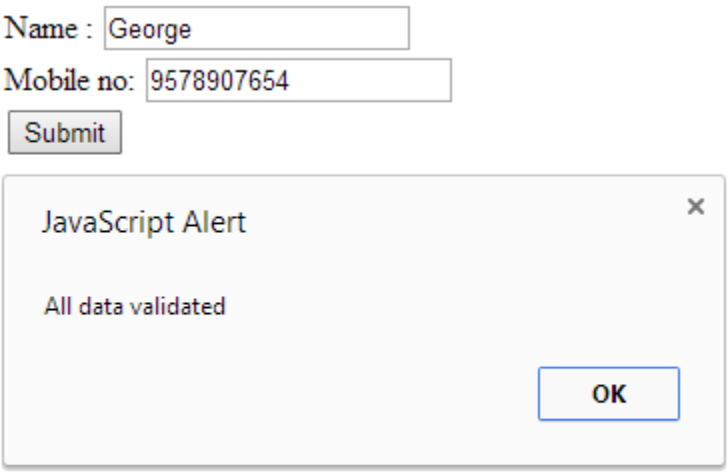
```html
<html>
<head>
</head>
<script >
    function validate(){
        var x=document.getElementById("name").value;
        var y=document.getElementById("number").value;

        if(x==""){
            alert('please enter your name');
            return false;
        }
        if(y==""){
            alert('please enter your number');
            return false;
        }
        if(isNaN(y)){
            alert('please enter numeric data as mobile number');
            return false;
        }
        if(y.length!=10){
            alert('please enter a valid mobile number of 10 digit');
            return false;
        }
        alert('All data validated')
       return true;
    }

</script>
<body>
<form action="/register.do" onsubmit="return validate()">
    Name    : <input type="text" id="name"><br>
    Mobile no: <input type="text" id="number"><br>
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

OUTPUT:


Name :
Mobile no:
Submit
JavaScript Alert
please enter your name
OK


Name : George
Mobile no: abcxyz
Submit
JavaScript Alert
please enter numeric data as mobile number
OK


Name : George
Mobile no: 123446
Submit
JavaScript Alert


Name : George
Mobile no: 9578907654
Submit
JavaScript Alert
All data validated
OK

Here first validations were did to find out whether the fields were left blank or not.
If any field was left blank, proper message would be shown.
Then the mobile number was validated for being a number first. When it is confirmed that its a number only, then its length was checked. If the length of the number is other than 10 then that means the number was not valid, so the error message was shown in the alert box.

Radio button validation

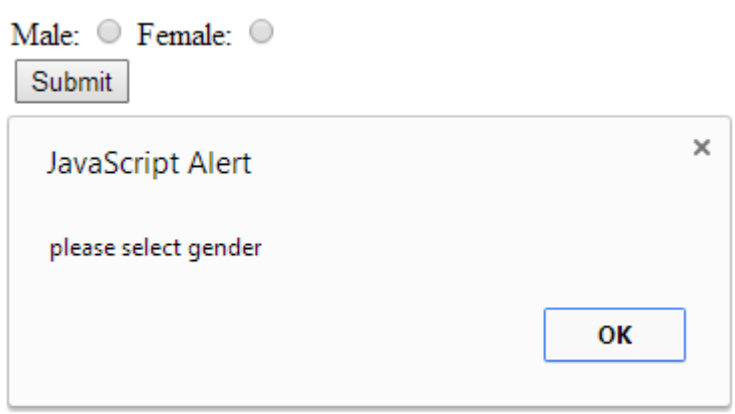
To check whether a radio button or check box is checked or not, simply first get the element by name, id or tag name and write the following conditional code:

if(x.checked), where x is the radio element or check box element. Thus we can validate each HTML element using JavaScript.

Sample code:

```
<html>
<head>
</head>
<script >
    function checkRadio(){
        var x=document.getElementsByName("gender")
        for(var i=0;i<x.length;i++){
            if(x[i].checked)
                return true;
        }
        alert('please select gender');
       return false;
    }

</script>
<body>
<form action="/register.do" >
    Male: <input type="radio" name="gender" value="male">
    Female: <input type="radio" name="gender" value="female"><br>
    <input type="submit" onclick="return checkRadio()">
</form>
</body>
</html>
```

OUTPUT:



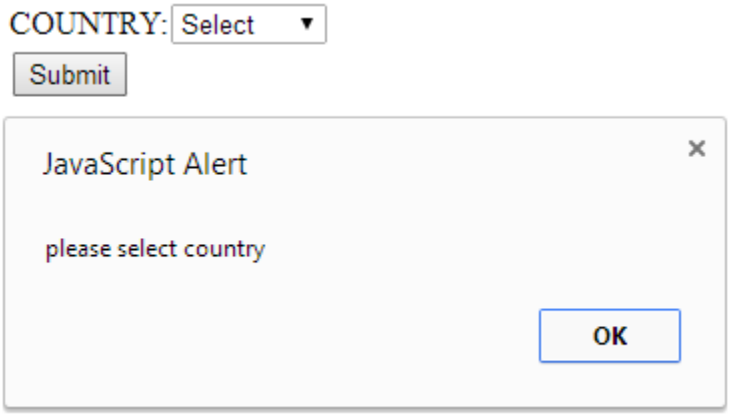
The above sample code that checks whether a radio button is checked or not.
As soon as JavaScript finds that the radio button is selected it return true and the form proceeds with submission otherwise false is returned and the form is not submitted. A corresponding message is also shown through the alert box.

Select box validation

Now let us see how to validate a select box.

```
<html>
<head>
</head>
<script >
    function checkRadio(){
        var x=document.myForm.country;
        if(x.selectedIndex==0)
            {
                alert('please select country');
                return false;
            }
        alert(x.value);
       return true;
    }

</script>
<body>
<form name="myForm" action="/register.do" >
    COUNTRY:<select name="country">
    <option value="">Select</option>
    <option value="France">France</option>
    <option value="Germany">Germany</option>
    <option value="India">India</option>
    <option value="Italy">Italy</option>
    <option value="Kenya">Kenya</option>

    </select><br>
    <input type="submit" onclick="return checkRadio()">
</form>
</body>
</html>
```

OUTPUT:



In the above example first the select element was fetched through the document object. Then the index of the selected option was checked. The index starts from 0. So after clicking the submit button if the index of selected option remains same, then that would mean the user did not select any option.

If the selected index is not 0 then the code would show the value of selected option through alert box. i.e “x.value” here would give the selected option value.

10.3 Reference Link

Java Script

http://www.w3schools.com/js/DEFAULT.asp